

# Desarrollo y ejecución de ETL con NiFi

Autor: Adrián Robles Arques

## Introducción

En este tercer Sprint, el último del módulo antes de realizar el proyecto final, vamos a aprender a utilizar la herramienta de Apache NiFi para diseñar y ejecutar un flujo de datos que conecte nuestros archivos locales con el sistema distribuido en HDFS. Para ello disponemos de datos de rutas aéreas tomadas desde OpenFlights, lo que completa nuestro trabajo con las tres fuentes de datos que ofrece esta plataforma.

## Consideraciones Previas

En este caso vamos a partir de un archivo en formato '.dat', que si revisamos el archivo o la página de OpenFlights vemos que cuenta con los siguientes datos:

- **Airline:** Código IATA o ICAO, formato de cadena.
- **Airline ID:** Identificador numérico de la aerolínea, entero.
- **Source airport:** Código IATA o ICAO del aeropuerto de origen.
- **Source airport ID:** Identificador del aeropuerto de origen, entero.
- **Destination airport:** Código IATA o ICAO del aeropuerto de destino, cadena.
- **Destination airport ID:** Identificador del aeropuerto de destino, entero.
- **Codeshare:** "Y" si el vuelo dispone de "codeshare", es decir, no es operado directamente por la aerolínea. Formato cadena.
- **Stops:** Número de paradas que realiza el avión hasta el destino final, entero.
- **Equipment:** Codigos de 3 letras que identifican cada avión que opera una ruta. Formato de cadena.

Ejemplo de datos de rutas, aquí podemos ver la cabecera y primeras líneas del documento:

```
1 |airline,airlineID,sourceAirport,sourceID,destinationAirport,destinationID,codeshare,stops,equipment
2 2B,410,AER,2965,KZN,2990,false,0,CR2
3 2B,410,ASF,2966,KZN,2990,false,0,CR2
4 2B,410,ASF,2966,MNU,2962,false,0,CR2
```

## Procedimiento

Para esta tercera fase voy a explicar, igual que en el caso anterior, los procesos nuevos, entendiendo que cuestiones como arrancar los servicios que ya se usaban previamente quedan cubiertas por los informes previos.

## Inicio de HDFS y comprobaciones iniciales

Como en el ejercicio anterior, comenzamos arrancando el sistema de datos distribuidos HDFS, que inicia el proceso local de los namenodes y datanodes. Después de arrancarlo podemos comprobar en la interfaz gráfica que se accede desde el puerto asignado del localhost que los ficheros previamente cargados siguen apareciendo.

```
bigdata@Big-Data:~$ cd /home/bigdata/hadoop-3.3.6/  
bigdata@Big-Data:~/hadoop-3.3.6$ sbin/start-dfs.sh  
Starting namenodes on [localhost]  
Starting datanodes  
Starting secondary namenodes [Big-Data]  
bigdata@Big-Data:~/hadoop-3.3.6$
```

Antes de continuar, procedo a revisar por pantalla la estructura de los datos con los que vamos a trabajar, para corroborar que la estructura de los datos es, efectivamente, la que hemos especificado previamente y que coincide con lo visto en el documento:

```
bigdata@Big-Data:~$ cat /home/bigdata/Descargas/rutas/routes.dat
```

```
ZH,4611,TYN,3369,SYX,4030,false,0,320  
ZH,4611,TYN,3369,SZX,3374,false,0,320 738  
ZH,4611,URC,3399,CGO,3375,false,0,320  
ZH,4611,URC,3399,CKG,3393,true,0,737  
ZH,4611,URC,3399,CTU,3395,true,0,320  
ZH,4611,URC,3399,HTN,3398,true,0,737  
ZH,4611,URC,3399,KHG,3397,true,0,737  
ZH,4611,URC,3399,PEK,3364,true,0,737 777 320  
ZH,4611,URC,3399,XIY,3379,true,0,737 320  
ZH,4611,URC,3399,YCU,6430,true,0,320  
ZH,4611,UYN,6368,CTU,3395,true,0,319  
ZH,4611,WNZ,6392,CAN,3370,false,0,738 320  
ZH,4611,WNZ,6392,CKG,3393,true,0,319  
ZH,4611,WNZ,6392,CTU,3395,true,0,320  
ZH,4611,WNZ,6392,KMG,3382,true,0,737  
ZH,4611,WNZ,6392,KWE,3394,true,0,319  
ZH,4611,WNZ,6392,PEK,3364,true,0,320 737 319  
ZH,4611,WNZ,6392,PVG,3406,true,0,319 320  
ZH,4611,WNZ,6392,SHE,4144,false,0,320  
ZH,4611,WNZ,6392,SYX,4030,false,0,320  
ZH,4611,WNZ,6392,SZX,3374,false,0,320  
ZH,4611,WNZ,6392,XIY,3379,true,0,320  
ZH,4611,WUA,6955,PEK,3364,true,0,737  
ZH,4611,WUH,3376,CAN,3370,false,0,320 738 737  
ZH,4611,WUH,3376,CGQ,4380,false,0,738  
ZH,4611,WUH,3376,CTU,3395,false,0,320  
ZH,4611,WUH,3376,HET,6345,false,0,320  
ZH,4611,WUH,3376,KMG,3382,false,0,738  
ZH,4611,WUH,3376,NNG,3373,false,0,738
```

Realizadas estas comprobaciones, continuamos con el arranque de Apache Nifi.

## Creación del flujo de datos con Apache Nifi

Para comenzar con esta parte del proceso, debemos ubicar la terminal en la carpeta donde esté instalado Nifi y arrancarlo con la instrucción apropiada en la terminal, en este caso 'bin/nifi.sh start':

```
bigdata@Big-Data:~$ cd /home/bigdata/nifi-1.23.2/
bigdata@Big-Data:~/nifi-1.23.2$ bin/nifi.sh start

Java home: /usr/lib/jvm/java-8-openjdk-amd64
NiFi home: /home/bigdata/nifi-1.23.2

Bootstrap Config File: /home/bigdata/nifi-1.23.2/conf/bootstrap.conf
```

Esto dará inicio a los sistemas de nifi, que también cuentan con interfaz gráfica accesible desde el navegador, como es el caso de HDFS. Sin embargo, aunque desde la terminal parezca que ha finalizado el proceso de arranque, aún puede tardar un minuto en ejecutarse completamente y permitirnos acceder a la interfaz.

Por otra parte, para visualizar los movimientos que realiza la herramienta, vamos a lanzar un comando que muestra los últimos 400 logs, con el comando 'tail -400f' y la referencia a la ubicación de los logs '/home/bigdata/nifi-1.23.2/logs/nifi-app.log':

```
bigdata@Big-Data:~/nifi-1.23.2$ tail -400f /home/bigdata/nifi-1.23.2/logs/nifi-app.log

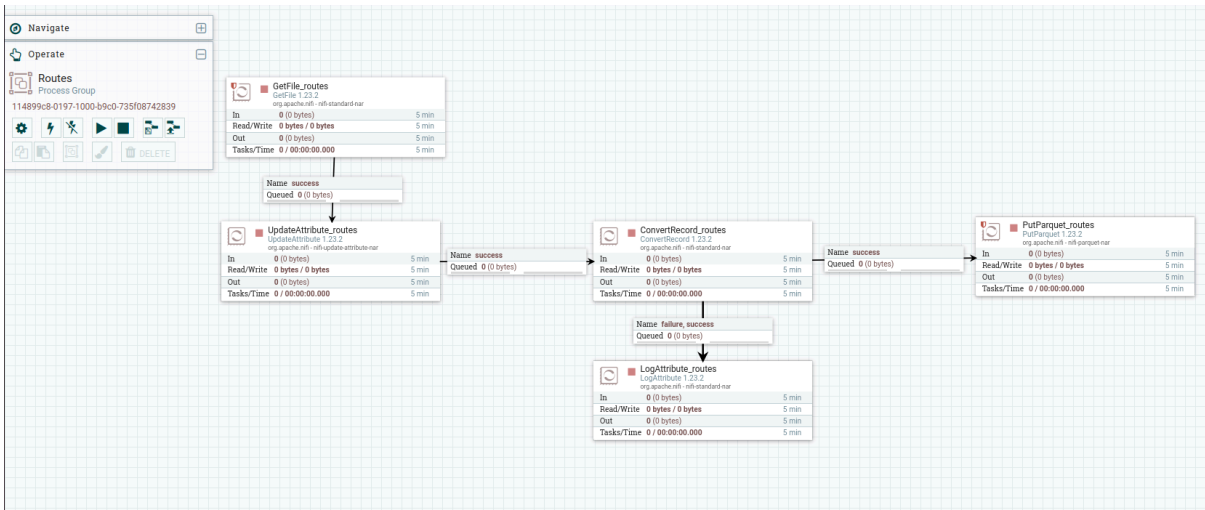
2025-05-27 12:25:59,137 INFO [main] o.eclipse.jetty.server.AbstractConnector Started ServerConnector@379ce046[SSL, (ssl, http/1.1)][127.0.0.1:8443]
2025-05-27 12:25:59,138 INFO [main] org.eclipse.jetty.server.Server Started @21319ms
2025-05-27 12:25:59,152 INFO [main] org.apache.nifi.web.server.JettyServer NiFi has started. The UI is available at the following URLs:
2025-05-27 12:25:59,152 INFO [main] org.apache.nifi.web.server.JettyServer https://127.0.0.1:8443/nifi
2025-05-27 12:25:59,154 INFO [main] org.apache.nifi.BootstrapListener Successfully initiated communication with Bootstrap
2025-05-27 12:25:59,154 INFO [main] org.apache.nifi.NiFi Started Application Controller in 13.694 seconds (13694342817 ns)
2025-05-27 12:26:17,379 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Initiating checkpoint of FlowFile Repository
2025-05-27 12:26:17,380 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Successfully checkpointed FlowFile Repository with 0 records in 0 milliseconds
2025-05-27 12:26:37,381 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Initiating checkpoint of FlowFile Repository
2025-05-27 12:26:37,381 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Successfully checkpointed FlowFile Repository with 0 records in 0 milliseconds
```

Una vez podamos acceder a la interfaz gráfica, voy a crear un grupo de procesadores, al que llamaré 'Routes', donde implementar el flujo de trabajo que necesitaremos para leer los datos de la carpeta local y transferirlos en formato parquet a HDFS.

The screenshot displays the Apache NiFi web interface. At the top, there's a navigation bar with the 'nifi' logo and several icons. Below this, a status bar shows various metrics: 0 nodes, 0/0 bytes, 0 processors, 0 queues, 0 errors, 0 alerts, and 1 warning. The main area shows a 'Routes' process group. On the left, there's a sidebar with 'Navigate' and 'Operate' tabs. The 'Routes' group is highlighted, showing its ID 'c1ceb197-0190-1000-2362-e65b09451ab0'. The 'Routes' group is currently empty, with a table showing 0 Queued, 0 In, 0 Read/Write, and 0 Out. The table also shows 0 bytes for each metric and a 5 min timeout. At the bottom, there's a row of status icons: a checkmark, a star, a plus, a minus, a question mark, and a warning icon, all with a count of 0.

| Metric     | Value | Unit            | Timeout |
|------------|-------|-----------------|---------|
| Queued     | 0     | (0 bytes)       |         |
| In         | 0     | (0 bytes) → 0   | 5 min   |
| Read/Write | 0     | bytes / 0 bytes | 5 min   |
| Out        | 0     | → 0 (0 bytes)   | 5 min   |

Para esta tarea vamos a seguir el diseño de flujo que se ve en la siguiente imagen:



Este flujo requiere de los siguientes procesadores:

- **GetFile:** Procesador que recaba el archivo ‘routes.dat’ de la carpeta de origen.

Configure Processor | GetFile 1.23.2

Stopped

SETTINGS

SCHEDULING

PROPERTIES

RELATIONSHIPS

COMMENTS

Required field

| Property               | Value                         |  |
|------------------------|-------------------------------|--|
| Input Directory        | /home/bigdata/Descargas/rutas |  |
| File Filter            | routes.dat                    |  |
| Path Filter            | No value set                  |  |
| Batch Size             | 10                            |  |
| Keep Source File       | false                         |  |
| Recurse Subdirectories | true                          |  |
| Polling Interval       | 0 sec                         |  |
| Ignore Hidden Files    | true                          |  |
| Minimum File Age       | 0 sec                         |  |
| Maximum File Age       | No value set                  |  |
| Minimum File Size      | 0 B                           |  |
| Maximum File Size      | No value set                  |  |

CANCEL

APPLY

- **UpdateAttributes:** Procesador encargado de añadir un identificador al nombre del archivo y cambiar la terminación a '.parquet'.

**Configure Processor** | UpdateAttribute 1.23.2

Stopped

SETTINGS SCHEDULING **PROPERTIES** RELATIONSHIPS COMMENTS

Required field 🔍 +

| Property                         | Value                                             |    |
|----------------------------------|---------------------------------------------------|----|
| Delete Attributes Expression     | No value set                                      |    |
| Store State                      | Do not store state                                |    |
| Stateful Variables Initial Value | No value set                                      |    |
| Cache Value Lookup Cache Size    | 100                                               |    |
| filename                         | \${UUID()}_\${filename:replace('dat', 'parquet')} | 🗑️ |
| schema.name                      | routes                                            | 🗑️ |

⚙️ ADVANCED CANCEL APPLY

- **ConvertRecord:** Procesador encargado de leer el fichero en su formato original y transformarlo en tipo parquet.

**Configure Processor** | ConvertRecord 1.23.2

Stopped

SETTINGS SCHEDULING **PROPERTIES** RELATIONSHIPS COMMENTS

Required field 🔍 +

| Property                      | Value                  |   |
|-------------------------------|------------------------|---|
| Record Reader                 | CSVReader              | → |
| Record Writer                 | ParquetRecordSetWriter | → |
| Include Zero Record FlowFiles | true                   |   |

CANCEL APPLY

- **LogAttribute:** Procesador que recibe metadatos del proceso de conversión y almacena los logs del proceso.

**Configure Processor** | LogAttribute 1.23.2

Stopped

SETTINGS SCHEDULING PROPERTIES RELATIONSHIPS COMMENTS

Required field

| Property                                   | Value              |
|--------------------------------------------|--------------------|
| Log Level                                  | info               |
| Log Payload                                | true               |
| Attributes to Log                          | No value set       |
| Attributes to Log by Regular Expression    | .*                 |
| Attributes to Ignore                       | No value set       |
| Attributes to Ignore by Regular Expression | No value set       |
| Log FlowFile Properties                    | true               |
| Output Format                              | Line per Attribute |
| Log prefix                                 | No value set       |
| Character Set                              | UTF-8              |

CANCEL APPLY

- **PutParquet:** Procesador final del flujo, encargado de almacenar la información cargada en el sistema HDFS usando el formato parquet.

**Configure Processor** | PutParquet 1.23.2

Invalid

SETTINGS SCHEDULING PROPERTIES RELATIONSHIPS COMMENTS

Required field

| Property                       | Value                                                     |
|--------------------------------|-----------------------------------------------------------|
| Hadoop Configuration Resources | /home/bigdata/hadoop-3.3.6/etc/core-site.xml, /home/bi... |
| Kerberos Credentials Service   | No value set                                              |
| Kerberos User Service          | No value set                                              |
| Kerberos Principal             | No value set                                              |
| Kerberos Keytab                | No value set                                              |
| Kerberos Password              | No value set                                              |
| Kerberos Relogin Period        | 4 hours                                                   |
| Additional Classpath Resources | No value set                                              |
| Record Reader                  | ParquetReader                                             |
| Directory                      | /nifi/routes                                              |
| Compression Type               | UNCOMPRESSED                                              |
| Overwrite Files                | true                                                      |

CANCEL APPLY

Además, para el funcionamiento de los distintos procesadores, vamos a necesitar los controladores siguientes:

- **AvroSchemaRegistry:** Dado que vamos a usar un consumidor de Avro, aportamos el esquema Registry para indicar el formato de los datos, de acuerdo con lo que hemos visto anteriormente al inspeccionar la tabla. Este controlador será usado a su vez por los controladores encargados de la lectura y escritura de datos.

**Configure Controller Service | AvroSchemaRegistry 1.23.2**

SETTINGS PROPERTIES COMMENTS

Required field

| Property             | Value |
|----------------------|-------|
| Validate Field Names | true  |
| routes               | {...} |

```

{
  "type": "record",
  "name": "routes",
  "fields": [
    { "name": "airline", "type": ["null", "string"] },
    { "name": "airlineID", "type": ["null", "int"] },
    { "name": "sourceAirport", "type": ["null", "string"] },
    { "name": "sourceID", "type": ["null", "int"] },
    { "name": "destinationAirport", "type": ["null", "string"] },
    { "name": "destinationID", "type": ["null", "int"] },
    { "name": "codeshare", "type": ["null", "string"] },
    { "name": "stops", "type": ["null", "int"] },
    { "name": "equipment", "type": ["null", "string"] }
  ]
}
  
```

CANCEL APPLY

- **CSVReader:** Aunque el fichero emplee la terminación genérica '.dat' el formateo de los datos es claramente similar a un CSV, por lo tanto usaremos un controlador indicado para este tipo de ficheros. Será utilizado por los procesadores **UpdateAttributes** y **ConvertRecord**.

**Configure Controller Service | CSVReader 1.23.2**

SETTINGS PROPERTIES COMMENTS

Required field

| Property                       | Value                       |
|--------------------------------|-----------------------------|
| Schema Access Strategy         | Use 'Schema Name' Property  |
| Schema Registry                | AvroSchemaRegistry_routes → |
| Schema Name                    | \${schema.name}             |
| Schema Version                 | No value set                |
| Schema Branch                  | No value set                |
| CSV Parser                     | Apache Commons CSV          |
| Date Format                    | No value set                |
| Time Format                    | No value set                |
| Timestamp Format               | No value set                |
| CSV Format                     | Custom Format               |
| Value Separator                | ,                           |
| Record Separator               | \n                          |
| Treat First Line as Header     | true                        |
| Ignore CSV Header Column Names | false                       |

CANCEL APPLY

- **ParquetRecordSetWriter**: Controlador empleado para escribir los datos introducidos en formato parquet. Será utilizado por el procesador de conversión **ConvertRecord**.

Configure Controller Service | ParquetRecordSetWriter 1.23.2

SETTINGS
PROPERTIES
COMMENTS

Required field

| Property                   | Value                       |
|----------------------------|-----------------------------|
| Schema Write Strategy      | Set 'schema.name' Attribute |
| Schema Cache               | No value set                |
| Schema Access Strategy     | Use 'Schema Name' Property  |
| Schema Registry            | AvroSchemaRegistry_routes   |
| Schema Name                | \${schema.name}             |
| Schema Version             | No value set                |
| Schema Branch              | No value set                |
| Cache Size                 | 1000                        |
| Compression Type           | UNCOMPRESSED                |
| Row Group Size             | No value set                |
| Page Size                  | No value set                |
| Dictionary Page Size       | No value set                |
| Max Padding Size           | No value set                |
| Enable Dictionary Encoding | No value set                |

CANCEL
APPLY

- **ParquetReader**: Controlador usado para leer los datos en formato parquet, que utilizará el procesador **PutParquet** para leer los datos y persistirlos en HDFS.

Controller Service Details | ParquetReader 1.23.2

SETTINGS
PROPERTIES
COMMENTS

Required field

| Property                | Value |
|-------------------------|-------|
| Avro Read Compatibility | true  |

OK



# Ingesta de datos a través de NiFi y carga en HDFS

Con el flujo de datos listo y la configuración de todos los elementos realizada, el siguiente paso es iniciar los controladores, que en primera instancia se generan deshabilitados. Además podemos apreciar las interdependencias pues tanto **CSVReader** como **ParquetRecordSetWriter** nos indican que no pueden funcionar sin activar previamente el **AvroSchemaRegistry**.

Routes Configuration

GENERAL

CONTROLLER SERVICES

|  | Name                      | Type                          | Bundle                                                 | State    | Scope  |  |
|--|---------------------------|-------------------------------|--------------------------------------------------------|----------|--------|--|
|  | AvroSchemaRegistry_routes | AvroSchemaRegistry 1.23.2     | org.apache.nifi-nifi-registry-nar                      | Disabled | Routes |  |
|  | CSVReader                 | CSVReader 1.23.2              | org.apache.nifi-nifi-record-serialization-services-nar | Invalid  | Routes |  |
|  | ParquetReader             | ParquetReader 1.23.2          | org.apache.nifi-nifi-parquet-nar                       | Disabled | Routes |  |
|  | ParquetRecordSetWriter    | ParquetRecordSetWriter 1.23.2 | org.apache.nifi-nifi-parquet-nar                       | Invalid  | Routes |  |

En cuanto habilitamos este primer controlador, vemos como los otros dos pasan a estar disponibles:

Routes Configuration

GENERAL

CONTROLLER SERVICES

| Name                                                                                                          | Type                          | Bundle                                                 | State                                                                                        | Scope  |                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------|-------------------------------|--------------------------------------------------------|----------------------------------------------------------------------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  AvroSchemaRegistry_routes | AvroSchemaRegistry 1.23.2     | org.apache.nifi-nifi-registry-nar                      |  Enabled  | Routes |   |
|  CSVReader                 | CSVReader 1.23.2              | org.apache.nifi-nifi-record-serialization-services-nar |  Disabled | Routes |   |
|  ParquetReader             | ParquetReader 1.23.2          | org.apache.nifi-nifi-parquet-nar                       |  Disabled | Routes |   |
|  ParquetRecordSetWriter    | ParquetRecordSetWriter 1.23.2 | org.apache.nifi-nifi-parquet-nar                       |  Disabled | Routes |   |

Arrancamos los tres controladores restantes:

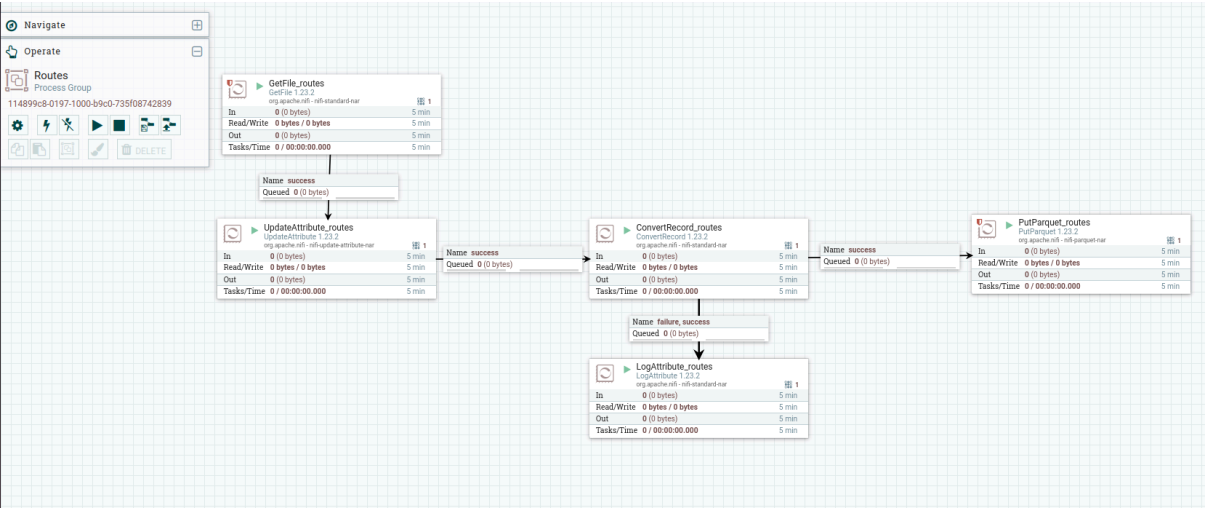
Routes Configuration

GENERAL

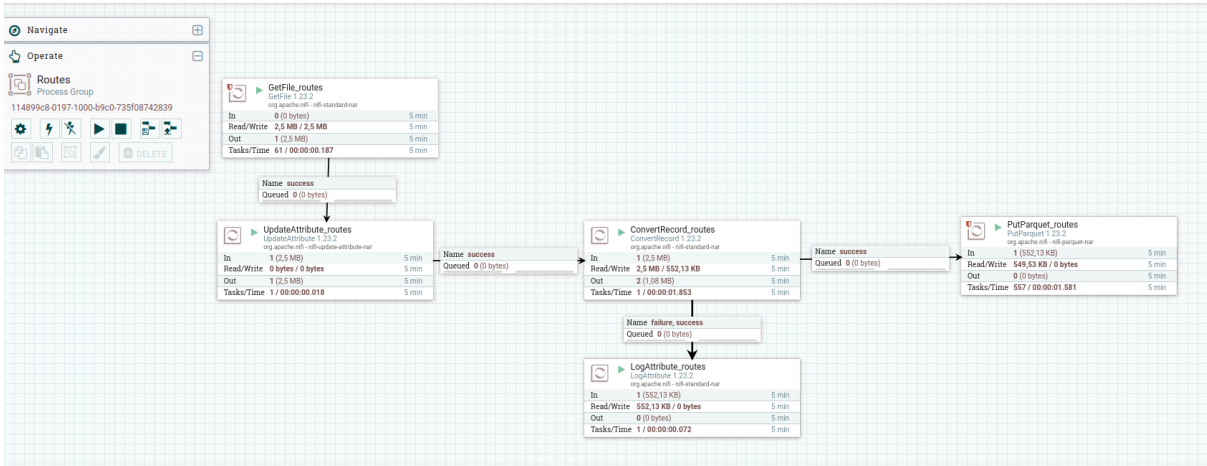
CONTROLLER SERVICES

| Name                                                                                                          | Type                          | Bundle                                                 | State                                                                                       | Scope  |                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------|-------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  AvroSchemaRegistry_routes | AvroSchemaRegistry 1.23.2     | org.apache.nifi-nifi-registry-nar                      |  Enabled | Routes |   |
|  CSVReader                 | CSVReader 1.23.2              | org.apache.nifi-nifi-record-serialization-services-nar |  Enabled | Routes |   |
|  ParquetReader             | ParquetReader 1.23.2          | org.apache.nifi-nifi-parquet-nar                       |  Enabled | Routes |   |
|  ParquetRecordSetWriter    | ParquetRecordSetWriter 1.23.2 | org.apache.nifi-nifi-parquet-nar                       |  Enabled | Routes |   |

El siguiente paso es iniciar el procesamiento de datos a través del flujo diseñado:



Una vez se inicia el procesamiento de datos, vemos cómo avanzan las tareas a lo largo de los diferentes procesadores. También podemos ver en consola, donde habíamos activado que se mostrasen los logs, cómo se muestran estas tareas que NiFi está llevando a cabo:



```
2025-05-27 12:59:17,551 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Initiating checkpoint of FlowFile Repository
2025-05-27 12:59:17,564 INFO [pool-7-thread-1] o.a.n.wall.SequentialAccessWriteAheadLog Checkpointed Write-Ahead Log with 0 Records and 0 Swap Files in 12 milliseconds (Stop-the-world time = 5 milliseconds), max Transaction ID 10
2025-05-27 12:59:17,564 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Successfully checkpointed FlowFile Repository with 0 records in 12 milliseconds
2025-05-27 12:59:18,775 INFO [FileSystemRepository Workers Thread-1] o.a.n.c.repository.FileSystemRepository Successfully archived 2 Resource Claims for Container default in 2 millis
2025-05-27 12:59:37,567 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Initiating checkpoint of FlowFile Repository
2025-05-27 12:59:37,567 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Successfully checkpointed FlowFile Repository with 0 records in 0 milliseconds
2025-05-27 12:59:54,697 INFO [Cleanup Archive for default] o.a.n.c.repository.FileSystemRepository Successfully deleted 2 files (3.04 MB) from archive
2025-05-27 12:59:54,697 INFO [Cleanup Archive for default] o.a.n.c.repository.FileSystemRepository Archive cleanup completed for container default; will now allow writing to this container. Bytes used = 20.64 GB, bytes free = 8.22 GB, capacity = 28.87 GB
2025-05-27 12:59:54,830 INFO [Write-Ahead Local State Provider Maintenance] org.wall.MinimalLockingWriteAheadLog org.wall.MinimalLockingWriteAheadLog@4fef5792 checkpointed with 23 Records and 0 Swap Files in 7 milliseconds (Stop-the-world time = 1 milliseconds, Clear Edit Logs time = 0 millis), max Transaction ID 72
2025-05-27 12:59:57,567 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Initiating checkpoint of FlowFile Repository
2025-05-27 12:59:57,568 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Successfully checkpointed FlowFile Repository with 0 records in 0 milliseconds
```

Finalmente podemos comprobar que se han creado las carpetas indicadas 'nifi/routes'\* en la interfaz gráfica de HDFS, y se han movido los datos a la carpeta de destino en formato parquet:

## Browse Directory

/

Go!

Show

25

entries

Search:

| <input type="checkbox"/> | Permission | Owner   | Group      | Size | Last Modified | Replication | Block Size | Name   |  |
|--------------------------|------------|---------|------------|------|---------------|-------------|------------|--------|--|
| <input type="checkbox"/> | drwxr-xr-x | bigdata | supergroup | 0 B  | May 21 09:28  | 0           | 0 B        | data   |  |
| <input type="checkbox"/> | drwxr-xr-x | bigdata | supergroup | 0 B  | May 20 14:47  | 0           | 0 B        | logs   |  |
| <input type="checkbox"/> | drwxr-xr-x | bigdata | supergroup | 0 B  | May 27 12:58  | 0           | 0 B        | nifi   |  |
| <input type="checkbox"/> | drwxr-xr-x | bigdata | supergroup | 0 B  | May 27 11:57  | 0           | 0 B        | test   |  |
| <input type="checkbox"/> | drwx-wx-wx | bigdata | supergroup | 0 B  | Jul 17 2024   | 0           | 0 B        | tmp    |  |
| <input type="checkbox"/> | drwxr-xr-x | bigdata | supergroup | 0 B  | May 20 14:47  | 0           | 0 B        | topics |  |
| <input type="checkbox"/> | drwxr-xr-x | bigdata | supergroup | 0 B  | May 20 14:57  | 0           | 0 B        | user   |  |

Showing 1 to 7 of 7 entries

Previous

1

Next

Hadoop, 2023.

\* La especificación de la ruta de guardado la configuré previamente a darme cuenta que se especificaba el uso de la ruta '/data/routes' en el Sprint, en mi caso creé una carpeta diferente para que contuviese sólo los datos cargados desde NiFi. Si esto fuera un problema para las siguientes fases del proyecto, puedo volver a cargar los datos en la otra ruta.

## Browse Directory

Show 25 entries

Search:

| <input type="checkbox"/> | Permission | Owner   | Group      | Size | Last Modified | Replication | Block Size | Name                   |  |
|--------------------------|------------|---------|------------|------|---------------|-------------|------------|------------------------|--|
| <input type="checkbox"/> | drwxr-xr-x | bigdata | supergroup | 0 B  | May 27 12:58  | 0           | 0 B        | <a href="#">routes</a> |  |

Showing 1 to 1 of 1 entries

Previous

1

Next

Hadoop, 2023.

## Browse Directory

Show 25 entries

Search:

| <input type="checkbox"/> | Permission | Owner   | Group      | Size      | Last Modified | Replication | Block Size | Name                                                                |  |
|--------------------------|------------|---------|------------|-----------|---------------|-------------|------------|---------------------------------------------------------------------|--|
| <input type="checkbox"/> | -rw-r--r-- | bigdata | supergroup | 552.13 KB | May 27 12:58  | 1           | 128 MB     | <a href="#">09fb1eec-1518-45f0-b08c-0ba204476b68_routes.parquet</a> |  |

Showing 1 to 1 of 1 entries

Previous

1

Next

Hadoop, 2023.

## Creación de la tabla de datos en Hive

Una vez los datos han sido procesados y cargados en HDFS, es momento de crear la tabla para operar con dicha información. Para ello comenzamos arrancando Hive.

```
Connected to: Apache Hive (version 3.1.3)
Driver: Hive JDBC (version 3.1.3)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 3.1.3 by Apache Hive
0: jdbc:hive2://>
```

Comprobamos las tablas que ya tenemos cargadas, para verificar que no existe la tabla ‘rutas’:

```
+-----+
|  tab_name  |
+-----+
| aerolineas |
| aeropuertos |
| fruits     |
| rutas      |
| shops      |
+-----+
```

A continuación procedemos a crear la tabla, siguiendo el esquema ya conocido:

```
0: jdbc:hive2://> CREATE EXTERNAL TABLE rutas(
. . . . . > airline string,
. . . . . > airlineID int,
. . . . . > sourceAirport string,
. . . . . > sourceID int,
. . . . . > destinationAirport string,
. . . . . > destinationID int,
. . . . . > codeshare string,
. . . . . > stops int,
. . . . . > equipment string)
. . . . . > STORED AS parquet
. . . . . > LOCATION '/nifi/routes/';
```

Para comprobar que la tabla se creó correctamente, vamos a mostrar las primeras 10 filas:

```
0: jdbc:hive2://> SELECT * FROM rutas LIMIT 10;
```

```
0: jdbc:hive2://> SELECT * FROM rutas LIMIT 10;
OK
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| rutas.airline | rutas.airlineid | rutas.sourceairport | rutas.sourceid | rutas.destinationairport | rutas.destinationid | rutas.codeshare | rutas.stops | rutas.equipment |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 2B            | 410              | AER                  | 2965           | KZN                      | 2990                | false           | 0           | CR2              |
| 2B            | 410              | ASF                  | 2966           | KZN                      | 2990                | false           | 0           | CR2              |
| 2B            | 410              | ASF                  | 2966           | MRV                      | 2962                | false           | 0           | CR2              |
| 2B            | 410              | CEK                  | 2968           | KZN                      | 2990                | false           | 0           | CR2              |
| 2B            | 410              | CEK                  | 2968           | OVB                      | 4078                | false           | 0           | CR2              |
| 2B            | 410              | DME                  | 4829           | KZN                      | 2990                | false           | 0           | CR2              |
| 2B            | 410              | DME                  | 4829           | NBC                      | 6969                | false           | 0           | CR2              |
| 2B            | 410              | DME                  | 4829           | TGK                      | -1                  | false           | 0           | CR2              |
| 2B            | 410              | DME                  | 4829           | UUA                      | 6160                | false           | 0           | CR2              |
| 2B            | 410              | EGO                  | 6156           | KGO                      | 2952                | false           | 0           | CR2              |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
10 rows selected (1,708 seconds)
```

Como vemos, la tabla se creó adecuadamente con el formato especificado y los datos son accesibles.

## Conclusiones

En este trabajo hemos aprendido a diseñar un flujo de datos empleando la herramienta de Apache Nifi, así como a configurarlo adecuadamente para gestionar diferentes tipos de archivos, usarlo para la gestión de logs y la persistencia de archivos en sistemas distribuidos de datos como HDFS.

Este trabajo es muy relevante para establecer un sistema complejo de datos distribuidos, dado que esta herramienta puede tomar el papel de un sistema integrador de procesos a lo largo de todo el recorrido de los datos.